Report

Orbital

Barnabas Tanczos

26 February 2025

1 Overview

The objective of this practical was to design and implement a video game in the Processing framework, including a point-mass physics engine that simulates forces and collisions in a 2D environment. All four levels of the requirements have been implemented in a modular game engine. The game *Orbital* demonstrates these mechanics with simple randomised physics-based puzzles.

2 Design

The primary design goal of the *Orbital* game was to showcase the various physics mechanics in the game engine, so that the player can play around with these components and explore their interactions in a light-hearted, relaxed environment. The result is a physics-based puzzle game with a series of short levels that showcase the various mechanics. Figure 1 shows a screenshot of a level of the game. It has a simple, cartoony aesthetic to match this theme. The player controls an alien that must manoeuvre around the level using both the environment (crates, springs, hanging balls, etc.) and their abilities (spawning blocks, freezing objects, teleporting, etc.).

Instead of a puzzle with a pre-determined solution given by the designer, each level poses a problem that the player can approach in different ways. Because the abilities they are given are randomised each time they attempt the level, the player is forced to improvise and adapt their strategy. The breadth of tools also increases the replayability, as the player can try new creative solutions to solve the same level. This approach is inspired by similar

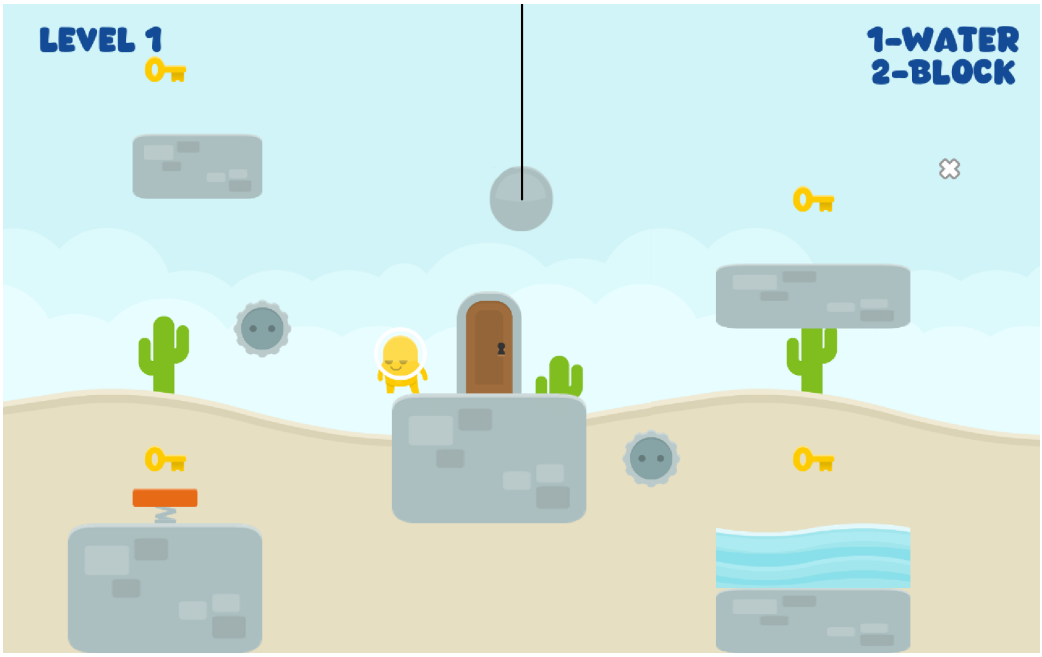


Figure 1: A level from the *Orbital* game.

puzzle games, such as *Mosa Lina* (2023), and more broadly by immersive sim games, such as *Deus Ex* (2000).

3 Implementation

3.1 Game Engine

The implementation of the game engine aims to provide an extensible, modular framework for creating game elements with various attached behaviours. This draws from the structure of other mainstream game engines like *Unity* or *Godot*. Figure 2 shows a UML diagram of the main relationships between the core components of the engine. All objects within the game, including characters, objects and backgrounds, are described by the fundamental type `GameObject`. This acts as a container for `Component` objects that determine the visuals and functionality of the element. This encapsulation allows for easy reuse of scripts and ensures each object only has the exact functionality it requires. A game object can have at most one `Component` of the same type. All game objects are stored in a hierarchical fashion in scenes. Each `Scene` contains an empty root game object, which has references to its child game objects, and so on in a recursive structure. This groups objects in a useful manner, providing a clear tree structure and facilitating operations on

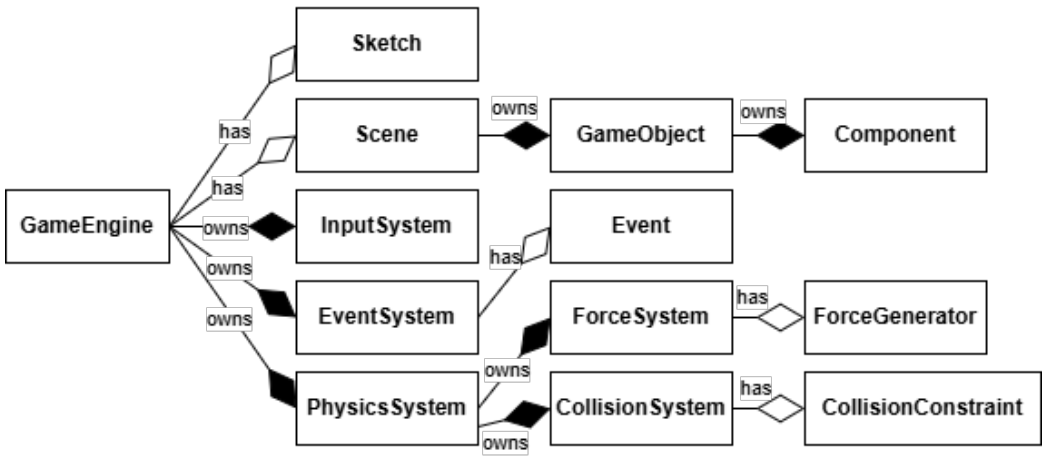


Figure 2: A UML diagram of the main relationships between the core game engine classes.

objects comprised of multiple sub-objects.

The `GameEngine` class is a singleton class that manages scenes and runs the core game loop every frame as it receives updates from the Processing app `Sketch`. There can be only one `Scene` active at a time. When a scene is activated, it first runs `initialize` to instantiate the hierarchy of game objects defined by the scene. Then, the scene runs the `start` method of the components of each game object in a recursive manner. This is used for setup in the components that only needs to happen once and may require other components to already exist. Subsequently, the engine runs the `update` method for both its components and its other systems, triggering behaviour that must be called on every frame. Finally, it runs `cleanUp` to remove objects and components that have been destroyed. This also resets all engine systems when a new scene is loaded. It is important that clean-up happens only at a defined point at the end of the frame, rather than instantaneously, as this prevents concurrent modification and null pointer exceptions that may occur if the hierarchy is modified during the execution of other logic.

Games define their own components by extending `BehaviorComponent`, but many fundamental components are integrated into the game engine. First among these is the `TransformComponent`, which tracks the position of an object both globally and relative to its parent object, enabling child objects to follow their parent. This is a prerequisite component for many others. Another important group of components are subclasses of `DrawableComponent`, which leverage the capabilities of Processing to display visuals including im-

ages, shapes, text and animations. There are also components for physics and collision, as described below. Beyond component updates, the game engine supports interaction through its `EventSystem`. The game can register a runnable method to handle an event whenever it is invoked, so it does not need to check conditions each frame. These events may be invoked by the `InputSystem` in response to user input, the `CollisionSystem` in response to collisions, or by individual behaviours.

3.2 Physics System

The engine physics system relies on the corresponding `PhysicsComponent`. This simulates a point mass at the transform component, and updates its position based on the motion equations for velocity and acceleration following the guidelines of Millington [1]. The translation is adjusted based on the current frame rate, to avoid it influencing the speed of objects. Additionally, damping is applied to the velocity each frame to prevent the object from speeding up due to numerical inaccuracies, and a minimum threshold is used to prevent jittering. The `PhysicsSystem` class itself registers all physics components and manages the force and collision subsystems.

Forces can be applied to a physics object to increase its acceleration. They are collected from different `ForceGenerator` sources in an accumulator before being applied together during the update. These force generators continually update the force of a paired object, as registered in the `ForceSystem`. This system maps force generators to their objects and triggers their updates each frame. It also tracks global force generators that update every physics component in the scene. `GravityGenerator` and `DragGenerator` are the most basic forces, the former producing a constant force in one direction and the latter producing a force opposing the current velocity. Meanwhile, the `SpringGenerator` produces a force dependent on the distance between two objects, pulling them together or pushing them apart towards a given rest length, with variants for only extension or compression. The `BuoyancyGenerator`, on the other hand, leverages the collision logic below to estimate the overlap between two objects, acting with a stronger force the more submerged an object is.

The other major component of the physics system, collision, is based around the `ColliderComponent`. This defines a shape that interacts with other collider shapes, either as a simple trigger detecting overlaps, or with more complex collision resolution behaviour. Two shapes are implemented: rectangles and squares. The `CollisionManager` class contains static methods that define whether two shapes are colliding through geometric calculations based on the boundary and centre points. Each collider is recorded

in the `CollisionSystem`, with different layers to enable grouping certain objects that can collide with each other. This system generates `Collision` instances and invokes the relevant events. Collisions are resolved by first calculating the normal depending on the combination of shapes. This can be used to obtain the separating velocity, which is adjusted so as to not vibrate on resting contact following [1]. Using this, impulse is calculated and applied to each object as they bounce off one another. The impulse also depends on the restitution, which is calculated from the elasticity property of both objects. Lastly, a translation is applied along the contact normal to correct any interpenetration that would result in objects clipping through others.

Constraints generate special instances of collisions based on the relationship between two objects. The `CollisionConstraint` class is the abstract version of this, with a method to add collisions each frame as updated by the `CollisionSystem`. The rod constraint implements this to force a fixed distance between two objects, using collisions to adjust their position to maintain the separation. Similarly, the cable constraint forces a maximum separation, but does not limit how close the objects may be.

3.3 Game

The *Orbital* game itself consists of a series of scenes (the start screen, end screen, and levels) as well as several custom behaviours. As each level consists of similar elements, they mostly call static methods from the `Assets` class to instantiate new game objects from a template with different parameters and positions. This class is also responsible for loading the images and other assets from the `data` folder (creative commons from [2] and [3]). Due to the existing framework and modular design, most game elements are simply created by attaching a few components to a game object and setting the relevant parameters. The new behaviours consist of mostly simple scripts for the level logic (keys, doors, hazards, etc.) as well as dynamically spawning and destroying new game objects for the various powers.

4 Evaluation

Overall, the design of the game engine was a success. The game object and components system is not only a robust modular design, but also drastically improved the ease of development of the actual game, particularly due to my experience with similar engines. It was interesting to develop a large general framework before experiencing it from the user side as I created a game within the engine. Frequently I would have to find compromises between the two, and this understanding leads me to better appreciate the processes of the

other game engines I use. I look forward to polishing and extending it over multiple projects to build a complete and self-contained library. The physics system in particular also achieved the goals set in the requirements, with a focus on extensibility for adding new force generators and collision constraints. Regarding the *Orbital* game itself, I believe it achieved its goal of providing a fun, light-hearted environment to showcase these physics elements. The visuals are particularly appealing with the sprites and animations, and there is a good variety of mechanics for a small game. In my opinion, there is a lot of potential to extend it into a more lengthy, complete experience.

However, due to time constraints, the number of elements in both the game engine and the game itself meant that both the physics system and the final game lack the depth and polish to make for the best player experience. For the physics system, the inclusion of a generic polygon collider would greatly improve the variety of shapes available, and adding in a rotation system would increase the realism significantly. Furthermore, the complexity of the collision resolution could be improved to eliminate cases where the physics does not perform as expected, such as with multiple objects squeezing against one another. Meanwhile, additional testing of the game would have reduced the occasional bugs arising from the interactions between the different powers and elements, while also improving the balance and difficulty curve between levels. Moreover, quality-of-life additions such as pausing and a soundtrack would have helped create a more professional game feel. With much of the framework already in place for the next project, I intend for it to be much more successful in this regard.

References

[1] Ian Millington. *Game Physics Engine Development*. Elsevier, Burlington, 2nd edition, 2010.

[2] Kenney. Platformer Pack Redux, 2015. Accessed: 2025-02-10 [online] https://kenney.nl/assets/ platformer-pack-redux.

[3] Khurasan. Softizen font, 2025. Accessed: 2025-02-10 [online] https://www.dafont.com/softizen.font.